

H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models

Zhenyu Zhang¹, Ying Sheng², Tianyi Zhou³, Tianlong Chen¹, Lianmin Zheng⁴, Ruisi Cai¹, Zhao Song⁵, Yuandong Tian⁶, Christopher Ré², Clark Barrett², Zhangyang Wang¹, Beidi Chen^{6,7}

¹University of Texas at Austin, ²Stanford University, ³University of California, San Diego, ⁴University of California, Berkeley, ⁵Adobe Research, ⁶Meta AI (FAIR), ⁷Carnegie Mellon University
{zhenyu.zhang, tianlong.chen, ruisi.cai, atlaswang}@utexas.edu, ying1123@stanford.edu, {chrismre, barrett}@cs.stanford.edu, t8zhou@ucsd.edu, lianminzheng@gmail.com, zsong@adobe.com, yuandong@meta.com, beidic@andrew.cmu.edu

Abstract

Large Language Models (LLMs), despite their recent impressive accomplishments, are notably cost-prohibitive to deploy, particularly for applications involving long-content generation, such as dialogue systems and story writing. Often, a large amount of transient state information, referred to as the KV cache, is stored in GPU memory in addition to model parameters, scaling linearly with the sequence length and batch size. In this paper, we introduce a novel approach for implementing the KV cache which significantly reduces its memory footprint. Our approach is based on the noteworthy observation that a small portion of tokens contributes most of the value when computing attention scores. We call these tokens *Heavy Hitters* (H₂). Through a comprehensive investigation, we find that (i) the emergence of H₂ is natural and strongly correlates with the frequent co-occurrence of tokens in the text, and (ii) removing them results in significant performance degradation. Based on these insights, we propose Heavy Hitter Oracle (H₂O), a KV cache eviction policy that dynamically retains a balance of recent and H₂ tokens. We formulate the KV cache eviction as a dynamic submodular problem and prove (under mild assumptions) a theoretical guarantee for our novel eviction algorithm which could help guide future work. We validate the accuracy of our algorithm with OPT, LLaMA, and GPT-NeoX across a wide range of tasks. Our implementation of H₂O with 20% heavy hitters improves the throughput over three leading inference systems DeepSpeed Zero-Inference, Hugging Face Accelerate, and FlexGen by up to 29×, 29×, and 3× on OPT-6.7B and OPT-30B. With the same batch size, H₂O can reduce the latency by up to 1.9×. The code is available at <https://github.com/FMInference/H2O>.

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable proficiency in a wide range of natural language processing applications such as content creation, summarization, and dialogue systems [1, 2, 3, 4]. However, their deployment is very costly. In addition to the widely-studied bottlenecks of model size and the quadratic cost of attention layers, the problem of the size of the KV cache, which stores the intermediate attention key and values during generation to avoid re-computation, is becoming increasingly prominent [5]. For instance, a 30 billion-parameter model with an input batch size of 128 and a sequence length of 1024 results in 180GB of KV cache. A natural approach is to limit its maximum size as is done in classical software or hardware caches [6]. However, it is challenging to reduce KV cache memory footprints in LLMs without accuracy drops.

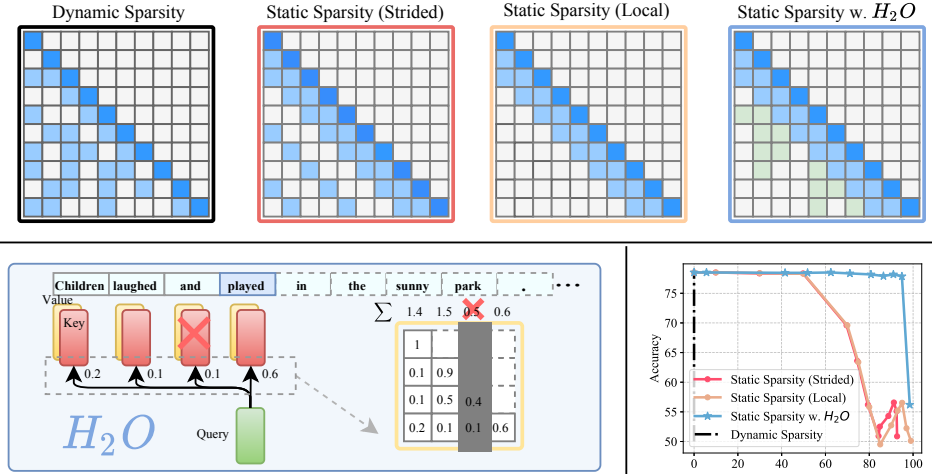


Figure 1: Upper plots illustrate symbolic plots of an attention map deploying different KV cache policies in LLM generation. Lower right: contrasts their accuracy-memory trade-off. Left: the overview of H_2O framework.

While there exists substantial literature on sparse attention approximation in training, they have not seen wide adoption for alleviating KV cache bottleneck. First, most existing methods, e.g., Reformer [7] and Flash Attention [8], are designed to overcome the quadratic memory required by attention mechanisms when modeling long sequences but still require a *large cache size*. Second, variants like sparse transformer [9], low-rank based transformers [10, 11] or multi-query attention [12, 13, 5] can reduce the cache size, but directly applying them on pre-trained LLMs for generation results in *high miss rates* and degrades the accuracy as shown in Figure 1. Finally, some recent advances such as gisting tokens [14] can learn to compress the KV cache for documents, but their *expensive eviction policies* are difficult to deploy during generation.

Therefore, an ideal KV cache should have (i) a *small cache size* to reduce memory footprint, (ii) a *low miss rate* to maintain the performance and long-content generation ability of LLMs, and (iii) a *low-cost eviction policy* to reduce the wall-clock time during generation. However, there are three technical challenges. First, it is not immediately clear whether the size of the KV cache can be restricted—each decoding step might, in principle, require access to all previous attention keys and values. Second, identifying an optimal eviction policy that maintains generation accuracy is a combinatorial problem¹. Finally, even if an optimal policy can be brute-forced, it is infeasible for deployment on real-world applications.

Fortunately, our preliminary exploration has yielded intriguing observations about the empirical properties of LLMs. These findings pave the way for the potential design of an efficient KV cache.

Sparsity for small cache size: We observe that even when trained densely, the attention matrices of LLMs are over 95% sparse at inference time (shown in Figure 2). This holds for a wide range of pre-trained LLMs. Therefore, only 5% of the KV cache is sufficient for decoding the same output token at each generation step, which suggests it may be possible to have up to a $20\times$ reduction in KV cache size without an accuracy drop.

Heavy-Hitters for low miss rate: We discover that the accumulated attention scores of all tokens in attention blocks adhere to a power-law distribution. It suggests that there exists a small set of influential tokens that are critical during generation, named heavy-hitters (H_2). H_2 provides an opportunity to step away from the combinatorial search problem and identify an eviction policy that maintains accuracy.

Greedy algorithm for low-cost policy: we surprisingly find that retaining the H_2 based on local statistics at each decoding step—summing the attention scores of only the preceding tokens—is as effective as considering the attention of future tokens (shown in Figure 2).

Based on the above, we first rigorously define the generative process of LLMs operating with a size-constrained KV cache in Section 2.1. Then we propose Heavy-Hitter Oracle (H_2O), a framework

¹Belady’s Algorithm is optimal for standard cache, but not necessarily for KV cache.

that exploits the properties of LLMs and uses simple, low-cost eviction policies that retrain the quality of LLMs throughout the generation process. Specifically,

- In Section 3, we explore the emergence of H_2 in attention, revealing their fundamental and critical roles: (i) H_2 exhibit a strong correlation of frequently co-occurring words in textual data; and (ii) removing H_2 completely damages the model’s functionality. We demonstrate that H_2 can largely lower the cache miss rate of the existing policies mentioned above. Theoretically, assuming the attention scheme is submodular, H_2 corresponds to a greedy algorithm and is therefore near-optimal.
- In Section 4, we present a greedy but low-cost variant of H_2 which is dynamically determined by the accumulated attention score at each decoding step. We formulate the eviction policy with greedy H_2 as a variant of dynamic submodular maximization. The analysis shows that it results in a similar generative process as the one using the H_2 eviction policy.

We perform extensive experiments on OPT, LLaMA, and GPT-NeoX on a single NVIDIA A100 (80GB) GPU to evaluate H_2O across a range of tasks from lm-eval-harness [15] and HELM [16]. We implement H_2O on top of FlexGen that can easily adapt different cache eviction techniques to produce a system with high-throughput inference. Performance experiments show our framework achieves $29\times$, $29\times$, $3\times$ higher throughputs compared to three leading inference systems, DeepSpeed Zero-Inference [17], Hugging Face Accelerate [18], and FlexGen [19] respectively. With the same batch size, H_2O achieves up to $1.9\times$ lower latency compare to FlexGen.

2 Related Work and Problem Setting

Efficient Inference of LLMs. The substantial parameter counts of large language models (LLMs) present significant challenges for inference. To overcome this limitation, previous efforts have employed model compression techniques with specific designs to achieve efficient LLM inference, such as the method described in [20, 21, 22], which employs one-shot pruning on LLMs, resulting in negligible performance degradation even without retraining. Additionally, alternative approaches explore quantization methods specifically tailored to LLMs, as discussed in [23, 24, 25, 26, 27, 28]. Also, CoLT5 [29] employs a token-wise conditional computation strategy to reduce the overall computation cost. These methods address efficient inference from orthogonal perspectives and can be organically integrated. The techniques investigated in this study are closely associated with pruning or sparsity but focus on a distinct inference bottleneck, namely, KV cache. One closely related work[30] utilizes a learnable mechanism that determines necessary tokens during inference but requires an extra fine-tuning process, which makes it less practical.

Sparse, Low-rank Attention Approx. The quadratic computational complexity of attention modules is one of the major bottlenecks of transformer inference [31]. Various efforts are devoted to addressing this challenge [7, 9, 10]. For example, Reformer [7] reduces the computational cost from quadratic to superlinear complexity via locality-sensitive hashing. Performer [10] employs positive orthogonal random features to approximate attention kernels. One relevant work, Sparse Transformer [9], introduces sparsity to reduce KV cache memory footprint and achieve an efficient attention mechanism, considered as our baseline in this paper. Moreover, SpAtten [32] utilizes accumulated attention scores to select important tokens for efficient attention inference while they don’t consider the variance of token importance across attention heads and layers. Comparison with SpAtten is detailed in Appendix C.9.

Caching. Caching, which plays a pivotal role in optimizing system performance, entails the development of effective eviction policies to handle frequently accessed data. Conventional approaches such as Least Recently Used and Least Frequently Used [33, 34] prioritize the recency and frequency of data access. And the design of KV cache encounters many similar challenges as traditional caching.

LLM Inference Breakdown. The generative procedure of LLMs encompasses two distinct phases: (i) the *prompt* phase, in which an input sequence is utilized to produce the KV cache (consisting of the key and value embeddings), similar to the forward pass employed during LLM training; and (ii) the *token generation* phase, which leverages and updates the KV cache to generate new tokens incrementally. Each generation step relies on the previously generated tokens. The primary focus of this paper is to enhance the efficiency of the KV cache in attention during the token generation phase, thereby accelerating LLM inference.